

LISTA DE EXERCÍCIOS
TYPESCRIPT
HERANÇA, POLIMORFISMO, INTERFACES & CLASSES ABSTRATAS

Orientações gerais

Todos os exercícios devem ser implementados em **TypeScript com `strict: true`**.

Regras gerais:

- usar identificadores em inglês;
- evitar `any`;
- evitar atributos públicos desnecessários;
- aplicar encapsulamento;
- escrever classes pequenas e coesas;
- justificar decisões de modelagem quando solicitado;
- não usar `if / switch` para descobrir o tipo concreto quando o objetivo for praticar polimorfismo;
- a partir do exercício 13, incluir testes unitários simples;
- a partir do exercício 23, incluir justificativa técnica curta;
- todas as soluções devem ser de autoria própria, sem uso de IA na implementação.

Parte I — Preparação para modelagem orientada a objetos

1. Identificação de classes e responsabilidades

Dado um pequeno enunciado sobre um sistema de biblioteca universitária, identifique:

- possíveis classes;
- atributos;
- métodos;
- responsabilidades;
- relações entre objetos;
- possíveis heranças;
- relações que **não** deveriam ser herança.

Não implementar código.

Objetivo: treinar análise antes da programação.

2. “É um tipo de” ou “tem um”?

Analise os pares abaixo e classifique a relação como herança, composição ou associação:

- `Student` **e** `Person`
- `Car` **e** `Engine`
- `Library` **e** `Book`
- `Professor` **e** `Course`
- `ShoppingCart` **e** `Product`
- `CheckingAccount` **e** `BankAccount`
- `Address` **e** `Person`
- `Report` **e** `PdfFile`

Para cada item, justifique em uma frase.

Objetivo: evitar uso inadequado de herança.

3. Refatoração inicial para classes

Receba uma solução procedural com funções e objetos literais para representar alunos, professores e técnicos.

Tarefa:

- identificar dados e comportamentos relacionados;
- criar classes adequadas;
- mover comportamentos para as classes correspondentes;

- evitar funções soltas quando o comportamento pertencer claramente a uma entidade.

Objetivo: transição do pensamento procedural para o pensamento orientado a objetos.

Parte II — Herança

Nesta etapa, o foco é compreender que uma subclasse representa uma especialização de uma classe mais geral.

4. Pessoas em uma universidade

Crie uma classe `Person` com:

- `name`
- `email`
- `getDescription()`

Crie as subclasses:

- `Student`
- `Professor`
- `AdministrativeTechnician`

Cada subclasse deve possuir pelo menos um atributo próprio.

Exemplo:

- `Student : registration`
- `Professor : academicTitle`
- `AdministrativeTechnician : department`

Objetivo: praticar herança simples.

5. Veículos

Crie uma classe `Vehicle` com:

- `brand`
- `model`
- `year`

Crie as subclasses:

- `Car`
- `Motorcycle`
- `Truck`

Cada subclasse deve acrescentar atributos próprios.

Exemplo:

- `Car : numberOfDoors`
- `Motorcycle : hasSidecar`
- `Truck : loadCapacityInKg`

Objetivo: identificar dados comuns e dados específicos.

6. Produtos de uma loja

Crie uma classe `Product` com:

- `description`
- `price`

Crie as subclasses:

- `Book`
- `ElectronicProduct`
- `FoodProduct`

Cada subclasse deve acrescentar informações específicas.

Exemplo:

- `Book : author`
- `ElectronicProduct : warrantyInMonths`
- `FoodProduct : expirationDate`

Objetivo: consolidar especialização de classes.

7. Contas bancárias

Crie uma classe `BankAccount` com:

- `holderName`
- `balance`
- `deposit()`
- `withdraw()`

Crie as subclasses:

- `CheckingAccount`
- `SavingsAccount`

A conta corrente deve possuir `overdraftLimit`.

A conta poupança deve possuir `interestRate`.

Objetivo: reaproveitar estado e comportamento comum.

8. Funcionários

Crie uma classe `Employee` com:

- `name`
- `registration`
- `baseSalary`

Crie as subclasses:

- `Manager`
- `Developer`
- `Intern`

Cada subclasse deve possuir uma informação específica.

Exemplo:

- `Manager : bonus`
- `Developer : programmingLanguage`
- `Intern : supervisorName`

Objetivo: modelar papéis especializados dentro de uma organização.

9. Animais

Crie uma classe `Animal` com:

- `name`
- `age`
- `species`

Crie as subclasses:

- `Dog`
- `Cat`
- `Bird`

Cada subclasse deve ter ao menos um atributo próprio.

Exemplo:

- `Dog : breed`
- `Cat : isDomestic`
- `Bird : wingspanInCm`

Objetivo: praticar hierarquia de classes em domínio simples.

10. Usuários de sistema

Crie uma classe `User` com:

- `id`
- `name`
- `email`
- `passwordHash`

Crie as subclasses:

- `StudentUser`
- `ProfessorUser`
- `AdminUser`

Cada tipo de usuário deve ter atributos próprios.

Objetivo: modelar usuários especializados em um sistema real.

11. Itens de biblioteca

Crie uma classe `LibraryItem` com:

- `title`
- `publicationYear`
- `locationCode`

Crie as subclasses:

- `Book`
- `Magazine`
- `Thesis`

Cada subclasse deve acrescentar informações próprias.

Objetivo: consolidar herança como mecanismo de especialização estrutural.

12. Herança inadequada

Analise a seguinte proposta de modelagem:

```
class ShoppingCart extends Array<Product> {}
```

Responda:

1. Por que essa herança é problemática?
2. `ShoppingCart` é realmente um tipo de `Array<Product>` ?
3. Que responsabilidades um carrinho possui além de armazenar produtos?
4. Como essa solução poderia ser melhorada usando composição?

Depois, implemente uma versão melhor com:

- `ShoppingCart`
- `CartItem`
- `Product`

Objetivo: compreender que herança não deve ser usada apenas para reaproveitar código.

Parte III — Polimorfismo

Nesta etapa, o aluno deve perceber que subclasses diferentes podem ser tratadas por meio do mesmo tipo base.

13. Sons de animais

A partir do exercício dos animais, adicione o método `makeSound()` em `Animal`.

Cada subclasse deve sobrescrever esse método.

Exemplo:

- `Dog` : "Au au"
- `Cat` : "Miau"
- `Bird` : "Piu piu"

Crie uma função que receba `Animal` e imprima o som.

Também crie testes para cada animal.

Objetivo: introduzir sobrescrita e polimorfismo.

14. Apresentação de usuários

A partir do exercício de usuários, adicione o método `getProfileDescription()` em `User`.

Cada subclasse deve retornar uma descrição diferente.

Crie uma função que receba `User[]` e gere as descrições de todos os usuários.

Objetivo: usar polimorfismo em coleção.

15. Cálculo de salário

A partir do exercício de funcionários, adicione o método `calculateMonthlyPay()` em `Employee`.

Cada subclasse deve calcular o salário de forma diferente.

Exemplo:

- `Manager` : salário base + bônus;
- `Developer` : salário base + adicional técnico;
- `Intern` : bolsa fixa.

Crie uma função que receba `Employee[]` e calcule a folha total.

Objetivo: evitar `if` por tipo de funcionário.

16. Preço final de produtos

A partir do exercício de produtos, adicione `calculateFinalPrice()` em `Product`.

Cada subclasse deve aplicar uma regra própria.

Exemplo:

- `Book` : desconto de 10%;
- `ElectronicProduct` : acréscimo de garantia;
- `FoodProduct` : desconto se estiver próximo do vencimento.

Crie uma função que receba `Product[]` e calcule o total da compra.

Objetivo: aplicar polimorfismo em regra de negócio.

17. Autonomia de veículos

A partir do exercício de veículos, adicione `calculateRange()` em `Vehicle`.

Cada subclasse deve calcular autonomia de forma diferente.

Exemplo:

- `Car` : consumo médio fixo;
- `Motorcycle` : consumo mais econômico;
- `Truck` : consumo depende da carga.

Crie uma função que receba `Vehicle[]` e exiba a autonomia de todos.

Objetivo: substituir condicionais por comportamento especializado.

18. Multa de itens de biblioteca

A partir do exercício de biblioteca, adicione `calculateLateFee(daysLate: number)` em `LibraryItem`.

Cada subclasse deve calcular a multa de forma diferente.

Exemplo:

- `Book` : R\$ 1,00 por dia;
- `Magazine` : R\$ 0,50 por dia;
- `Thesis` : R\$ 2,00 por dia.

Crie uma função que receba `LibraryItem[]` e calcule a multa total.

Objetivo: consolidar polimorfismo com comportamento especializado.

19. Personagens de jogo

Crie uma classe `GameCharacter` com:

- `name`
- `healthPoints`
- `attack()`
- `receiveDamage(amount: number)`

Crie as subclasses:

- `Warrior`
- `Mage`
- `Archer`

Cada subclasse deve sobrescrever `attack()` com cálculo diferente de dano.

Crie uma função que receba `GameCharacter[]` e execute o ataque de todos.

Objetivo: aplicar polimorfismo em domínio lúdico.

20. Relatório de folha de pagamento

A partir de `Employee`, crie o método `generateReportLine()`.

Cada subclasse deve gerar uma linha de relatório diferente.

Crie uma classe `PayrollReport` que receba `Employee[]` e gere um relatório textual.

Objetivo: usar polimorfismo para delegar detalhes às subclasses.

21. Refatoração de condicionais por polimorfismo

Receba uma versão inicial com lógica semelhante a:

```
if (employee.type === "manager") {  
    // calculate manager salary  
} else if (employee.type === "developer") {  
    // calculate developer salary  
} else if (employee.type === "intern") {  
    // calculate intern salary  
}
```

Tarefa:

- criar uma hierarquia com `Employee`, `Manager`, `Developer` e `Intern`;
- mover o cálculo para as subclasses;
- remover o `if / switch` por tipo;
- criar testes antes e depois da refatoração.

Objetivo: perceber o valor prático do polimorfismo.

22. Processamento de pedidos

Crie uma classe `Order` com `number`, `customerName` e `amount`.

Crie subclasses:

- `PhysicalProductOrder`
- `DigitalProductOrder`
- `SubscriptionOrder`

Cada tipo deve sobrescrever `process()`.

Exemplo:

- pedido físico gera separação para entrega;
- pedido digital gera liberação de acesso;
- assinatura gera ativação de período.

Crie `OrderProcessor` que receba `Order[]` e processe todos sem verificar o tipo concreto.

Objetivo: aplicar polimorfismo em fluxo de aplicação.

Parte IV — Interfaces

Nesta etapa, o foco é entender interface como contrato de comportamento. Interface representa uma capacidade, não necessariamente uma relação natural de herança.

23. Objetos imprimíveis

Crie a interface `Printable` com:

```
interface Printable {  
    print(): string;  
}
```

Implemente em:

- `Invoice`
- `Certificate`
- `Report`

Crie `PrinterService` que receba `Printable[]`.

Inclua testes para o serviço.

Justifique por que `Printable` deve ser interface.

Objetivo: introduzir interface como contrato.

24. Canais de notificação

Crie a interface `NotificationChannel` com:

```
interface NotificationChannel {
    send(message: string): void;
}
```

Implemente:

- `EmailChannel`
- `SmsChannel`
- `PushChannel`

Crie `NotificationService` dependente apenas de `NotificationChannel`.

Depois adicione `ConsoleChannel` sem alterar `NotificationService`.

Objetivo: reduzir acoplamento com classes concretas.

25. Exportadores de dados

Crie a interface `DataExporter` com:

```
interface DataExporter {
    export(data: Student[]): string;
}
```

Implemente:

- `CsvExporter`
- `JsonExporter`
- `PlainTextExporter`

Crie `StudentExportService`.

O serviço não deve usar `if` para descobrir o formato.

Objetivo: aplicar interface para variar estratégia de saída.

26. Meios de pagamento

Crie a interface `PaymentMethod` com:

```
interface PaymentMethod {
    pay(amount: number): PaymentResult;
}
```

Implemente:

- `CreditCardPayment`
- `PixPayment`
- `BankSlipPayment`

Crie `CheckoutService` dependente apenas da interface.

Objetivo: praticar programação orientada a contratos.

27. Políticas de desconto

Crie a interface `DiscountPolicy` com:


```
interface DiscountPolicy {
    calculateDiscount(amount: number): number;
}
```

Implemente:

- NoDiscountPolicy
- StudentDiscountPolicy
- LoyalCustomerDiscountPolicy
- PromotionalDiscountPolicy

Crie `Order` recebendo uma política de desconto.

Objetivo: trocar comportamento sem alterar a classe principal.

28. Capacidades de animais

Crie as interfaces:

```
interface Flyable {
    fly(): string;
}

interface Swimmable {
    swim(): string;
}

interface Walkable {
    walk(): string;
}
```

Crie classes:

- Duck
- Eagle
- Penguin
- Fish
- Dog

Cada classe deve implementar apenas as capacidades adequadas.

Objetivo: mostrar que interfaces representam capacidades, não hierarquias naturais.

29. Conteúdos de curso

Crie as interfaces:

```
interface Gradable {
    calculateGrade(): number;
}

interface Downloadable {
    download(): string;
}
```

Crie classes:

- VideoLesson
- TextMaterial
- Quiz
- ProjectActivity

Nem todo conteúdo deve ser avaliável.

Nem todo conteúdo deve ser baixável.

Objetivo: evitar herança artificial usando interfaces.

30. Reservas de espaços

Crie as interfaces:

```
interface Reservable {
    reserve(): void;
}

interface RequiresApproval {
    requestApproval(): void;
}
```

Crie classes:

- Classroom
- Laboratory
- Auditorium
- MeetingRoom

Alguns espaços podem ser reservados diretamente.
Outros exigem aprovação prévia.

Objetivo: modelar regras por contrato de comportamento.

31. Mini-framework de validação

Crie a interface genérica:

```
interface ValidationRule<T> {
    validate(value: T): ValidationResult;
}
```

Implemente regras como:

- RequiredStringRule
- MinLengthRule
- EmailFormatRule
- PositiveNumberRule
- DateInFutureRule

Crie `Validator<T>` que receba várias regras e retorne uma lista de erros.

Objetivo: usar interface, generics e polimorfismo de forma extensível.

32. Análise: quando usar interface?

Para cada caso abaixo, diga se uma interface seria adequada e justifique:

- Printable
- AcademicDocument
- PaymentMethod
- Vehicle
- Downloadable
- User
- NotificationChannel
- Student

A resposta deve indicar se o conceito representa:

- uma capacidade;
- um contrato;
- uma entidade concreta;
- uma especialização;
- ou uma base comum com estado.

Objetivo: consolidar a diferença entre contrato e entidade.

Parte V — Classes abstratas

Nesta etapa, o aluno deve compreender que uma classe abstrata é útil quando existe uma base comum com implementação parcial, mas que ainda depende de subclasses para completar comportamentos.

33. Documentos acadêmicos

Crie uma classe abstrata `AcademicDocument` com:

- `title`
- `author`
- método concreto `generateHeader()`
- método abstrato `generateBody()`
- método concreto `generateFullText()`

Crie as subclasses:

- `ResearchProject`
- `InternshipReport`
- `FinalPaper`

Objetivo: usar classe abstrata com comportamento comum e comportamento obrigatório.

34. Relatórios institucionais

Crie uma classe abstrata `InstitutionalReport` com:

- `title`
- `createdAt`
- método concreto `generate()`
- método abstrato `generateContent()`

Crie as subclasses:

- `StudentReport`
- `CourseReport`
- `InfrastructureReport`

Objetivo: aplicar um esqueleto comum de geração de relatório.

35. Procedimentos de atendimento

Crie uma classe abstrata `ServiceProcedure` .

Ela deve possuir o método concreto `execute()` .

Esse método deve chamar:

- `validate()`
- `perform()`
- `finish()`

O método `perform()` deve ser abstrato.

Crie:

- `MedicalConsultation`
- `DentalProcedure`
- `NursingProcedure`

Objetivo: aplicar uma forma introdutória do padrão Template Method.

36. Cálculo de impostos

Crie uma classe abstrata `Tax` com:

- `name`
- **método abstrato** `calculate(amount: number): number`

Crie:

- `ServiceTax`
- `ProductTax`
- `ImportTax`

Crie `TaxCalculator`, que recebe `Tax[]`.

Objetivo: representar uma família de regras com identidade comum.

37. Eventos acadêmicos

Crie uma classe abstrata `AcademicEvent` com:

- `title`
- `date`
- `capacity`
- **método concreto** `hasAvailableSeats()`
- **método abstrato** `calculateWorkloadInHours()`

Crie as subclasses:

- `Lecture`
- `Workshop`
- `ShortCourse`
- `RoundTable`

Objetivo: combinar estado comum, método concreto e método abstrato.

38. Importadores de arquivos

Crie uma classe abstrata `FileImporter` com o método concreto `import()`.

Esse método deve executar o fluxo:

1. validar arquivo;
2. ler conteúdo;
3. converter conteúdo;
4. retornar registros importados.

Crie métodos abstratos quando necessário, como:

- `parseContent()`
- `validateFormat()`

Subclasses:

- `CsvStudentImporter`
- `JsonStudentImporter`
- `TxtStudentImporter`

Objetivo: aplicar classe abstrata para padronizar fluxo com variações específicas.

39. Interface, classe abstrata ou classe concreta?

Analise os conceitos abaixo e escolha a melhor opção:

- interface;
- classe abstrata;
- classe concreta.

Conceitos:

1. `Printable`
2. `AcademicDocument`
3. `PaymentMethod`

4. User
5. Vehicle
6. Downloadable
7. InstitutionalReport
8. NotificationChannel
9. Tax
10. Student

Para cada item, justifique tecnicamente.

Critérios de análise:

- existe estado comum?
- existe implementação comum?
- o conceito pode ser instanciado?
- representa capacidade?
- representa uma família de objetos?
- representa uma entidade concreta?

Objetivo: consolidar a distinção entre interface e classe abstrata.

Parte VI — Integração final

40. Mini-projeto final: sistema de eventos acadêmicos

Modele um sistema simples para organização de eventos acadêmicos.

Use obrigatoriamente:

- pelo menos uma hierarquia de herança;
- pelo menos um uso de polimorfismo com array ou serviço;
- pelo menos duas interfaces;
- pelo menos uma classe abstrata;
- testes unitários;
- justificativa curta de modelagem;
- diagrama simples de classes.

Elementos sugeridos:

- AcademicEvent
- Lecture
- Workshop
- ShortCourse
- RoundTable
- Participant
- Registration
- Certificate
- AttendanceRecord

Interfaces sugeridas:

```
interface Registrable {
    registerParticipant(participant: Participant): void;
}

interface Certifiable {
    issueCertificate(participant: Participant): Certificate;
}

interface RequiresAttendanceControl {
    registerAttendance(participant: Participant): void;
}
```

Regras mínimas:

- cada tipo de evento calcula carga horária de forma própria;
- nem todo evento gera certificado;
- nem todo evento exige controle de frequência;
- inscrição deve respeitar limite de vagas;
- certificado só deve ser emitido se as regras forem atendidas;

- `RegistrationService`, `AttendanceService` e `CertificateService` devem ter responsabilidades separadas;
- evitar classes grandes e métodos longos;
- evitar `if / switch` por tipo concreto quando houver alternativa polimórfica.

Objetivo: integrar herança, polimorfismo, interfaces, classes abstratas, coesão, baixo acoplamento e testabilidade.